

Kapitel 12

Zeiger

12.1 Adressen und Zeiger

Eine Variable besitzt sowohl einen Speicherplatz als auch einen Wert. Beide benötigen wir je nach Kontext. Sehen wir uns das für eine Wertzuweisung $v = e$; im einzelnen an:

- Im Ausdruck auf der rechten Seite benötigen wir als Operanden *Werte*; wenn als Operanden Variablen angegeben sind, müssen die Werte der Variablen verwendet werden, nicht ihre Adressen.
- Links vom Zuweisungsoperator benötigen wir die *Adresse* der Variablen, in der der Wert des Ausdrucks gespeichert werden soll.

Die Theorie spricht von *Dereferenzierung*: Der Name einer Variablen repräsentiert die Adresse der Variablen; durch Dereferenzierung erhält man ihren Wert. In den Programmiersprachen ist es allgemein *nicht* üblich, dies auch formal nachzuvollziehen, weil ohnehin Verwechslungen nicht zu befürchten sind:

- Weil als Operand in einem Ausdruck meist nur der Wert einer Variablen Sinn macht, nicht jedoch ihre Adresse, wird die Dereferenzierung automatisch vorgenommen.
- Weil links vom Zuweisungsoperator einer Wertzuweisung nur eine Adresse Sinn macht, nicht jedoch der Wert einer Variablen, unterbleibt die Dereferenzierung.

Man muss aber noch weiter differenzieren: Bei den kombinierten Wertzuweisungen und den Inkrement- und Dekrementoperatoren muss sowohl dereferenziert als auch die Adresse verwendet werden. Betrachten wir als Beispiel die beiden gleichwertigen Anweisungen

```
i++;  
i += 1;
```

Zunächst wird *i* (automatisch) dereferenziert, um den *Wert* von *i* zu erhalten. Dann wird dieser Wert um 1 erhöht. Schließlich wird die *Adresse* von *i* verwendet, um das Resultat der Operation zu speichern. Aber auch dieses erfolgt, wie gesehen, automatisch und damit letztlich für den Programmierer verdeckt.

Nicht in jedem Falle ist die automatische Dereferenzierung erwünscht. Gesehen haben wir das zum Beispiel bei der Funktion `scanf`: Da die Funktion Werte in Variablen speichern soll, ist ihr mit *Werten* nicht gedient – sie benötigt *Adressen*! Wir haben auch folgendes bereits gesehen (jetzt in der neuen Terminologie): Der *Adressoperator* `&` verhindert, dass Variablen dereferenziert werden! Einsetzen kann man diesen Operator prinzipiell überall – nur macht er ohne weiteres an den meisten Stellen keinen Sinn.

Die Umkehrung des Adressoperators `&` ist der *Dereferenzierungsoperator* `*`: Er bewirkt, dass sein Operand dereferenziert wird, oder, anders formuliert: Er bewirkt, dass der Wert

seines Operanden dereferenziert und der dabei resultierende Wert als Wert des Ausdrucks betrachtet wird.

Der Dereferenzierungsoperator hebt den Adressoperator in seiner Wirkung offensichtlich auf. Wir sehen uns dazu das Beispiel

```
int i = 42, j;
j = *&i;
```

an: Der Teilausdruck `&i` liefert die Adresse von `i` als Resultat. Durch den Operator `*` wird der Wert des Teilausdrucks `&i` dereferenziert und liefert den Wert von `i` als Resultat. Die Wertzuweisung entspricht insgesamt also `j = i`; – nur umständlich ausgedrückt.

In C bezeichnet man Adressen als *Zeiger*. Ein Ausdruck wie `&i` wird entsprechend als *Zeigerausdruck* bezeichnet.

12.2 Zeiger als Funktionsparameter (I)

Wie bereits erwähnt erfolgt bei Funktionsaufrufen ein „call by value“. Das bedeutet, nur der *Wert* des Aufrufarguments – z.B. einer Variable `i` – wird übergeben. Demzufolge kann ein Funktionsaufruf nicht auf die Variable `i` zurückwirken, ihren Wert also nicht verändern. Dies wird weiter unten noch einmal an einem Beispiel erläutert.

Bei Feldern als Funktionsparameter ist das Verhalten scheinbar anders. Wie schon erwähnt, wird der Name eines Feldes (ohne Indizes) von C als Zeigerkonstante betrachtet. Bedeutung hatte das für uns im Zusammenhang mit Funktionen: Sehen wir als Parameter ein Feld vor und übergeben wir als Argument ein Feld durch seinen Namen, so haben wir in der Funktion direkten Zugriff auf das Feld. Entsprechend war es uns möglich, nicht nur auf die Werte der Komponenten zuzugreifen, sondern auch die Werte nachhaltig zu verändern. Wie lässt sich dasselbe für einfache Variablen erreichen?

Wie der Aufruf einer solchen Funktion auszusehen hat, wissen wir bereits von `scanf`: Anstelle des Namens der Variablen, etwa `v`, tragen wir den Ausdruck `&v` ein. Das bewirkt, wie wir eben noch einmal gesehen haben, dass die Dereferenzierung von `v` unterbleibt und entsprechend der Zeiger auf den Speicherplatz von `v` anstelle des Wertes von `v` an die Funktion übergeben wird.

Bei der Einführung der Funktionen wurde schon erwähnt, dass auch in der Definition einer Funktion Änderungen erforderlich sind, wenn Zeiger übergeben werden. Diese Änderungen betreffen zwei Punkte:

- Im Funktionsheader muss, sowohl in der Deklaration als auch in der Definition, markiert werden, dass ein Parameter ein Zeiger ist. Das geschieht, indem dem Namen des Parameters ein Stern (`*`) vorangestellt wird.
- Im Funktionsrumpf muss dem Namen des Parameters in der Regel ebenfalls an jeder Stelle ein Stern vorangestellt und damit explizit dereferenziert werden.

Als Beispiel betrachten wir eine „Trivialfunktion“: Sie soll nur den Wert einer Variablen der rufenden Funktion um 1 erhöhen. Bisher hätte ein Versuch wohl so ausgesehen:

```
void inkrement1(int i) {
    i = i + 1;
}
```

So funktioniert das nicht: `inkrement1` erhält eine Kopie des Wertes von `i`. Wenn, wie hier geschehen, `inkrement1` diese Kopie verändert, bleibt das für die rufende Funktion ohne Folgen. Wir müssen deshalb

```
void inkrement2(int *i) {  
    *i = *i + 1;  
}
```

schreiben: `i` selbst ist jetzt eine *Zeigervariable*. Das zeigt die Parametervereinbarung `int *i` an. Die (automatische) Dereferenzierung von `i` im Zuge der Auswertung eines Ausdrucks liefert wie bisher den Wert von `i` – nur ist das jetzt ein Zeiger auf eine `int`-Variable und nicht mehr ein `int`-Wert. Wollen wir auf den `int`-Wert zugreifen, müssen wir also noch einmal explizit dereferenzieren. Und das geschieht gerade durch `*i`.

Rufen wir die Funktion jetzt durch

```
j = 1;  
inkrement2(&j);
```

so hat `j` nach dem Aufruf den Wert 2.

12.3 Zeigervariablen

Nicht nur Parameter von Funktionen dürfen Zeigertypen erhalten, sondern auch beliebige andere Variablen. Dabei ist der Formalismus derselbe: In der Vereinbarung wird dem Namen ein Stern vorangestellt. Sollen mehrere Variablen in einer Zeile definiert werden, so wird für jede einzeln durch das vorangestellte Sternchen entschieden, ob es sich um eine Zeigervariable oder eine normale Variable des Typs handelt. So werden durch

```
int i, *k, j;
```

zwei `int`-Variablen `i` und `j` deklariert und eine Zeigervariable `k`, deren Wert einen Zeiger auf eine `int`-Variable repräsentiert. Für `i` und `j` gibt es keine Unterschiede zur bisherigen Verwendung. Mit `k` dürfen wir jetzt zum Beispiel

```
k = 10;
```

keinesfalls mehr schreiben: Die Anweisung würde ja bedeuten, dass der Wert 10 in der Folge als Zeiger auf eine `int`-Variable zu interpretieren wäre – es gibt jedoch für den C-Programmierer keine Möglichkeit, das sicherzustellen, weil die Zuordnung von Adressen zu den Variablen eines Programms *ausschließlich* dem Compiler (und Linker) bzw. den Funktionen zur dynamischen Speicherbereitstellung der Standardbibliothek vorbehalten ist. Eine zulässige Anweisungsfolge wäre dagegen

```
int i, j, *k;  
  
k = i < j ? &i : &j;  
*k = 10;
```

Sehen wir uns an, was im einzelnen passiert:

- Der Wert, der `k` letztlich zugeordnet wird, ist entweder `&i` oder `&j`. Beide Werte sind Zeiger auf eine `int`-Variable und stimmen damit im Typ mit `k` überein. Entsprechend ist die Wertzuweisung zulässig und speichert in `k` einen Zeiger auf eine `int`-Variable.
- Bei der Dereferenzierung von `k` resultiert der Zeiger auf eine `int`-Variable, hier also aufgrund der Wertzuweisung entweder der Zeiger auf `i` oder `j`. Entsprechend wird der `int`-Wert entweder in die `int`-Variable `i` oder in die `int`-Variable `j` übertragen.

C verlangt auch für Zeiger eine Typbindung. So haben wir bislang in den Beispielen „Zeiger auf `int`“ verwendet. Das hat Vorteile: Im letzten Beispiel etwa „weiß“ der Compiler, dass `*k` den Typ `int` besitzt. Er kann damit bei Bedarf den Typ eines zuzuweisenden Wertes oder beim Zugriff mit `*k` den Typ `int` „geeignet“ umwandeln. Die Wertzuweisung

```
*k = 7.567;
```

würde so ein Abschneiden der Stellen hinter dem Punkt bewirken. Hätte `k` nur den Typ „Zeiger“, wäre eine solche automatische Umwandlung prinzipiell nicht möglich.

12.4 Zeiger und const

Das Zusammenspiel zwischen Zeigervariablen und dem Schlüsselwort `const` ist komplex. Bei der Deklaration von Zeigervariablen kann das Schlüsselwort `const` an verschiedenen Stellen stehen. Steht es links von `*`, so bedeutet es, dass die von der Zeigervariablen referenzierten Werte konstant sind. Die Werte, auf die der Zeiger verweist, können also nicht geändert werden. Man kann der Zeigervariablen jedoch jederzeit einen neuen Wert zuweisen. An welcher Stelle `const` links von `*` steht, hat keine Bedeutung. Im Folgenden bezeichnen `i` und `j` Variablen vom Typ `int`.

```
const int *p = &i;    /* oder: int const *p = &i; */

*p = 2;  /* FEHLER: referenzierter Wert konstant */
p = &j;  /* erlaubt */
```

Eine andere Bedeutung hat `const` jedoch rechts von `*`. Dort bewirkt es, dass die Zeigervariable selbst konstant ist. D.h. einmal initialisiert kann man ihr keinen neuen Adresswert zuweisen. Der von ihr referenzierte Wert kann jedoch geändert werden.

```
int * const p = &i;

*p = 3;  /* erlaubt */
p = &j;  /* FEHLER: Zeigervariable konstant */
```

Eine Kombination aus beiden Fällen ist ebenfalls möglich. Es gelten dann entsprechend beide Einschränkungen für die Zeigervariable.

```
int const * const p = &i;

*p = 4;  /* FEHLER: referenzierter Wert konstant */
p = &j;  /* FEHLER: Zeigervariable konstant */
```

Man kann sich die Bedeutung leicht merken, indem man die Deklaration von rechts nach links liest. Im zweiten Beispiel kann man so lesen: „`p` ist ein *konstanter* Zeiger auf *int*-Werte“.

Beim Umgang mit Zeigern und `const` ist besondere Aufmerksamkeit erforderlich. Durch Zuweisungen kann das `const`-Attribut versehentlich verloren gehen. Der Standard sieht das nicht als Fehler an und der Compiler produzieren entsprechend meist nur eine Warnung.

```
const int k = 42;
const int *p = &k;
int *q = &k;          /* WARNUNG: const Attribut */
                      /* wird abgestreift */
*q = 43;              /* k hat nun den Wert 43!! */

q = p;                /* WARNUNG: ... */
```

Das Schlüsselwort `const` kann natürlich auch für Funktionsargumente mit der gleichen Bedeutung wie bei Variablendeklarationen verwendet werden. In der Praxis wird man es dort sogar wesentlich häufiger antreffen.

Zeiger als Parameter von Funktionen werden in C verwendet, um „*call by reference*“ zu simulieren. So wird das Zeigerargument selbst zwar kopiert, durch Dereferenzierung erhält man aber direkten Zugriff auf die eigentlichen Daten und kann diese ändern. Oft möchte man mit einem Zeigerargument aber nur das unter Umständen aufwändige automatische Kopieren der Argumente umgehen, da sich bei großen Datenstrukturen die Parameterübergabe mit „*call by value*“ negativ auf die Laufzeit auswirken könnte. Das Schlüsselwort **const** im Prototyp einer Funktion ist dann ein wertvoller Hinweis, dass die Daten nicht verändert werden.

12.5 Zeiger und Felder (I)

In C besteht ein enger Zusammenhang zwischen Zeigern und Feldern. Wir haben auch bereits gesehen:

- C betrachtet den Namen eines Feldes ohne Indizes als Zeigerkonstante. Ebenso ist der Name eines Feldes ein Zeiger, wenn er mit weniger Indizes angegeben wird als das Feld Dimensionen hat.
- Als Zeiger repräsentiert ein Feldname die Adresse der ersten Komponente des Vektors.
- Der Zugriff auf eine Feldkomponente erfolgt durch Berechnung der Speicherabbildungsfunktion für ihre Indizes.

Verfolgt man diese Prinzipien konsequent weiter, so wird es möglich, Formulierungen wie

```
int v[10], *p;
...
p = &v[0];
p = v;
```

zu verwenden.

Dass die erste der beiden Zuweisungen zulässig ist, haben wir oben schon gesehen: Zunächst wird die Speicherabbildungsfunktion berechnet, weil die eckigen Klammern als Operatoren höchste Priorität besitzen. Dann unterbleibt wegen des Adressoperators die Dereferenzierung, der Wert des Ausdruck `&v[0]` ist ein Zeiger und der Typ des Ausdrucks ist „Zeiger auf eine `int`-Variable“.

Die zweite der beiden Zuweisungen ist die Folgerung aus den eben angestellten Überlegungen: Der Ausdruck `v` ist der Zeiger auf die erste Komponente des Feldes `v`.

Der Standard legt fest, dass Variablenamen (einschließlich Feldkomponenten) automatisch dereferenziert werden, Feldnamen dagegen nicht. Laut Standard ist auch die Anweisung

```
p = &v;
```

nicht verboten, aber der Typ des Ausdrucks `&v` ist nur „Zeiger“ und nicht „Zeiger auf eine `int`-Variable“.

Wir können das Beispiel nun fortsetzen. Erlaubt ist jetzt zum Beispiel

```
int v[10], *p;

p = v;
*p = 17 + 4;
```

Der Wert des Ausdrucks `17 + 4` wird ausgewertet und sein Resultat in der Variablen gespeichert, auf die der Wert der Zeigervariablen `p` zeigt, also in `v[0]`. Die letzte Zuweisung entspricht also gerade

```
v[0] = 17 + 4;
```

Zur Erinnerung noch einmal: Namen von Feldern sind Zeigerkonstanten. Das schließt Zuweisungen wie

```
int v[10], *p, i;

v = p;
v = &i;
```

von vornherein aus, auch wenn in beiden Zuweisungen die Typen der Ausdrücke links und rechts vom Zuweisungsoperator übereinstimmen. Von der Logik her verlangen sie dasselbe wie die Wertzuweisung

```
7 = i;
```

12.6 Zeigerarithmetik

Was wir bislang über Zeiger gelernt haben, könnte als Sammlung von Formalien erscheinen, die zumindest auf den ersten Blick keinen Nutzen haben – abgesehen einmal von der Möglichkeit, „call by reference“ zu simulieren. Wirklich interessant werden Zeiger tatsächlich auch erst durch zwei zusätzliche Möglichkeiten:

- Da Zeigervariablen ihre Werte während der Ausführungszeit eines Programms ändern können, kann man mit ihnen auf Speicher zugreifen, der erst während der Ausführung des Programms bereitgestellt wird. Diese Möglichkeit kennen praktisch alle modernen Programmiersprachen; das Stichwort ist „Speicher auf dem Heap“. Die Besprechung dieser Möglichkeit wird auf Kapitel 13 verschoben.
- Die andere Möglichkeit ist eine Eigenart von C: Mit Zeigern kann man – in gewissen Grenzen – auch rechnen. Und damit müssen wir uns jetzt beschäftigen.

Der Ausgangspunkt sind erneut die Felder: Schreiben wir etwa

```
int v[10];

v[7] = 9;
```

so bewirkt `v[7]` ja gerade, dass auf das Element zugegriffen wird, das den Abstand 7 vom Anfang des Feldes hat. Anders formuliert: Der Zeiger auf den Anfang des Feldes, repräsentiert durch den Namen `v`, wird um 7 erhöht und der resultierende Wert für den Zugriff verwendet. Wir dürften das auch als

```
*(v + 7) = 9;
```

hinschreiben; die Wirkung entspricht gerade der eben gegebenen Beschreibung. Die Klammern sind dabei *unbedingt* nötig, da der Dereferenzierungsoperator wie alle unären Operatoren eine höhere Priorität besitzt als die binären Operatoren.

Entsprechend kann man nun natürlich auch

```
int v[10], *p;

p = v + 7;
*p = 9;
```

schreiben – und sind damit bei „echter“ Zeigerarithmetik angelangt: Was hindert uns daran, etwa

```
p++;  
p += 3;
```

zu schreiben?

Damit kommt man zu formal ganz anderen Formulierungen von Programmen, die auf Feldern operieren. Im letzten Abschnitt hatten wir zum Beispiel zur Berechnung der Summe der Komponenten eines Vektors die Funktion

```
float vektorsumme(const float v[], int laenge) {  
    float summe = 0;  
    while (--laenge >= 0)  
        summe += v[laenge];  
    return summe;  
}
```

formuliert. Die C-typische Formulierung wäre aber

```
float vektorsumme(const float *v, int laenge) {  
    float summe = 0;  
    while (laenge--)  
        summe += *v++;  
    return summe;  
}
```

Einige Anmerkungen hierzu:

- Die Parameterdefinitionen `const float v[]` und `const float *v` sind gleichwertig:
 - Beide Definitionen besagen, dass das Argument ein Zeiger auf eine `float`-Variable mit konstantem Wert sein muss/wird.
 - Dass die Variable als erste Komponente eines Vektors zu betrachten ist, ergibt sich bei der ersten Definition direkt aus der Definition, während es bei der zweiten Definition aus der Logik der Funktion entnommen werden muss. Wenn `v` keinen Vektor bezeichnet, sollte man deshalb zur Verdeutlichung `const float *const v` schreiben.
- Beim Aufruf wird der Wert des ersten Arguments in die lokale Variable `v` der Funktion kopiert, so dass die Änderung von `v` durch `v++` auch dann zulässig ist, wenn das Argument eine Konstante ist (hier: der Name eines Vektors).
- Beim Ausdruck `*v++` nutzen wir, dass die unären Operatoren alle dieselbe Priorität besitzen und dass sie ggf. von rechts nach links abgearbeitet werden. Der Ausdruck ist also gleichwertig mit `*(v++)`.

12.7 Operationen mit Zeigern

Arithmetik mit Zeigern ist nur eingeschränkt möglich. Wir müssen uns deshalb einmal im Zusammenhang ansehen, welche Operationen für Zeiger überhaupt zulässig sind:

- Zeigern können Werte zugewiesen werden. Wir haben bereits verschiedentlich Wertzuweisungen für Zeiger gesehen.
- Ganze Zahlen können auf Zeiger addiert und von ihnen subtrahiert werden. Welche Operatoren man dazu verwendet, ist gleichgültig. So sind für eine Zeigervariable `p` und einen `int`-Ausdruck `i` die Ausdrücke

```

p++
p + i
p--
p - i

```

sämtlich (formal) zulässig.

Über das, was logisch sinnvoll ist, müssen wir uns noch besonders Gedanken machen.

- Die Differenz von zwei (typgleichen) Zeigern kann gebildet werden. Das funktioniert so: Es wird unterstellt, dass beide Zeiger auf Komponenten ein und desselben Vektors zeigen. Die Differenz entspricht dann gerade der Differenz der Indizes der beiden Komponenten.

Eine solche Differenz besitzt einen ganzzahligen, vorzeichenbehafteten Typ – welcher es im einzelnen ist, muss durch die Deklaration des Typs `ptrdiff_t` in der Standard-Headerdatei `stddef.h` festgelegt werden.

Die Anweisungsfolge

```

int v[10], *p, *q;
ptrdiff_t d1, d2;

p = &v[3];
q = &v[7];
d1 = p - q;
d2 = q - p;

```

liefert so in `d1` den Wert `-4` und in `d2` den Wert `+4`.

Die Addition zweier Zeiger ist dagegen unzulässig. Was sollte für ein Feld `v` auch `v + v` bedeuten? Diesem Umstand ist besonders Rechnung zu tragen, wenn Terme vereinfacht werden sollen. Seien z.B. `start` und `ende` Zeiger auf den Anfang und das Ende eines Feldes. Soll nun ein Zeiger auf die Mitte des Feldes berechnet werden, so kann das folgendermaßen geschehen:

```

int v[20], *start = v, *ende = v + 20, *mitte;
mitte = start + (ende - start)/2;

```

Wendet man nun einfache mathematische Termumformungen an, könnte man auf folgende kürzere Schreibweise kommen

```

mitte = (start + ende)/2;    /* nicht zulaessig */

```

die für beliebige Ganzzahlwerte sicher richtig wäre, und mit einer Operation weniger auskommt. Für Zeiger ist dieser Ausdruck jedoch nicht zulässig.

- Zeiger können verglichen werden. Dafür sind alle 6 Vergleichsoperatoren zulässig. Der Ablauf entspricht der Bildung der Differenz von zwei Zeigern: Es wird unterstellt, dass beide Zeiger auf Komponenten ein und desselben Vektors zeigen. Der Vergleich liefert dasselbe Resultat wie ein Vergleich der Indizes.

Die Berechnung der Vektorsumme könnten wir zum Beispiel auch so formulieren:

```

float vektorsumme(const float *v, int laenge) {
    float summe = 0, *p = v + laenge;
    while (v < p)
        summe += *v++;
    return summe;
}

```


Auf den ersten Blick scheint hier `v + laenge` nicht erlaubt, weil das Resultat nicht auf eine Komponente des Feldes zeigt, sondern direkt *hinter* das Feld. Dies lässt der Standard allerdings ausdrücklich zu. Diese Technik wird häufig verwendet, wenn Bereiche aus einem Feld durch zwei Zeiger begrenzt werden sollen. Der erste Zeiger wird auf das Anfangselement gesetzt, während der zweite genau hinter das letzte Element des Bereichs zeigt. Das hat den Vorteil, dass die Differenz der beiden Zeiger gerade die Anzahl der Elemente im Bereich angibt und `for`-Schleifen wie gewohnt mit `<` als Abbruchbedingung formuliert werden können.

- Es gibt einen Makro `NULL`, der den *Nullzeiger* definiert. Der Wert dieses Makros muss von den Implementatoren so gewählt werden, dass kein „echter“ Zeiger ihn haben kann. Er erlaubt die Markierung von Zeigervariablen als „ohne Wert“. Definiert wird der Makro `NULL` in den Standard-Headerdateien `stddef.h`, `stdio.h` und `string.h`. Trotz der Bezeichnung sollte man sich nicht darauf verlassen, dass der Nullzeiger den Wert 0 hat und stattdessen immer `NULL` verwenden. So wird auch sofort klar, dass es sich um einen Zeiger handelt.

12.8 Zeiger als Parameter (II)

Beim Beispiel der Vektorsumme haben wir bereits gesehen, dass es bei Funktionen letztlich gleichgültig ist, ob wir einen Parameter als Vektor oder als Zeiger deklarieren. Zeiger sind allerdings typischer für C – und erlauben vielfach extrem kompakte Formulierungen von Funktionen. Zur Demonstration sollen zwei Beispiele dienen. Dass beide Beispiele Strings bearbeiten, ist nicht nur Zufall.

Erstes Beispiel: Zu kopieren ist ein String. Unter Verwendung von Feldern können wir schreiben

```
void strcpy1(char s[], const char t[]) {
    int i = 0;
    while (s[i] = t[i])
        i++;
}
```

Zeiger erlauben die Formulierung

```
void strcpy2(char *s, const char *t) {
    while (*s++ = *t++)
        ;
}
```

Zweites Beispiel: Zwei Strings sind zu vergleichen. Der Funktionswert soll kleiner, gleich oder größer als Null sein, je nachdem, ob der erste String (lexikographisch) kleiner, gleich oder größer als der zweite String ist. Für Vektoren können wir die Lösung so formulieren:

```
int strcmp1(const char s[], const char t[]) {
    int i = 0;
    while (s[i] == t[i] && s[i])
        i++;
    return s[i] - t[i];
}
```

Mit Zeigern kann die Lösung so aussehen:

```
int strcmp2(const char *s, const char *t) {
    while (*s == *t && *s)
        s++, t++;
}
```

```
    return *s - *t;
}
```

Ganz groß sind die Unterschiede nicht, wie man sieht. Was bei den Zeiger-Varianten entfällt, sind die lokalen Indexvariablen. Im übrigen sind, wie wir bereits gesehen haben, die Formulierung des Funktionsheaders und des Funktionsrumpfes voneinander unabhängig. Wir können also ohne weiteres im Header eine Vektor- und im Rumpf eine Zeigerformulierung verwenden und umgekehrt.

12.9 Probleme mit Zeigern

Mit Zeigern kann man mancherlei Unsinn treiben. Bei der Beschreibung der Operationen mit Zeigern wurde zweimal vorausgesetzt, dass die Zeiger in einem speziellen Zusammenhang zueinander stehen müssen. Nämlich bei der Berechnung der Differenz und beim Vergleich von zwei Zeigern. Sehen wir uns dazu ein Beispiel an:

```
int v1[10], v2[20];

if (v1 < v2)
    ...
```

Als Programmierer hat man keinen Einfluss darauf, wie der Compiler die Variablen (und Felder) eines Programms im Speicher anordnet! Ob die Bedingung `v1 < v2` in unserem Beispiel *wahr* oder *falsch* ist, hängt aber gerade davon ab, ob das Feld `v1` vom Compiler *vor* oder *hinter* dem Feld `v2` in den Speicher gelegt wurde. Ein Vergleich von zwei Zeigern macht in der Regel nur dann Sinn, wenn beide Zeiger in das gleiche Feld zeigen.

Auch ein falscher Indexzugriff oder allgemein das Dereferenzieren eines ungültigen Zeigers kann zu Problemen führen und ist häufig Ursache von Programmabstürzen. Das untersuchen wir anhand eines Beispiels:

```
int v[10], w[50], i, *p = v;
/* Berechnung von i */
v[i] = 44;
*(p + i) = 44;
```

Welchen Wert `i` hat, liegt erst fest, wenn die Wertzuweisungen ausgeführt werden. Allerdings:

- Es ist für den Compiler natürlich ein Kleines, zusätzliche Maschinenbefehle zu erzeugen, mit denen geprüft wird, ob der Wert von `i` zwischen 0 und 9 liegt, bevor die erste Wertzuweisung ausgeführt wird.
- Bei der zweiten Wertzuweisung ließen sich solche Prüfungen allenfalls mit erheblichem Aufwand erzeugen: Worauf `p` zeigt, wird erst während der Ausführung des Programms festgelegt. Der zulässige Wertebereich im Beispiel kann also 0 bis 9 sein; er kann aber auch 0 bis 49 sein, wenn `p` auf den Anfang von `w` „umgesetzt“ wurde. Wenn `p` durch Zeigerarithmetik verändert wurde, wird es endgültig unmöglich, den zulässigen Wertebereich für `i` zu bestimmen.

Dass die Auswertung von Zeigerausdrücken immer vernünftige Werte ergibt, ist ausschließlich in die Verantwortung des Programmierers gestellt. Verstöße wirken sich in der Regel als Abstürze des Programms aus sobald ein ungültiger Zeigerausdruck dereferenziert wird.

Besonders „schöne“ Möglichkeiten eröffnen aber auch die Castoperatoren. Zeiger sind ja an Typen gebunden. Schreiben wir etwa

```
int *ip;
float *fp;
...
fp = ip;
```

so wird der Compiler hoffentlich eine Fehlermeldung ausgeben. Wir könnten jetzt auf die Idee kommen, das durch

```
fp = (float *) ip;
```

zu „reparieren“. Um die Abläufe etwas zu verdeutlichen, muss das Beispiel etwas erweitert werden:

```
int i, *ip = &i;
float *fp;
...
fp = (float *) ip;
*fp = 6.5f;
printf ("%d\n", i);
```

Letztlich haben wir hier den Typ von `i` (temporär) geändert: Mit `*fp` greifen wir auf den Speicherplatz der `int`-Variablen `i` zu. Bei der Wertzuweisung wäre also eigentlich eine Umwandlung des Typs des Ausdrucks `6.5f` von `float` in `int` erforderlich – diese Umwandlung unterbleibt aber, weil `*fp` als Zeiger auf `float` definiert ist. Statt dessen wird die Bitfolge unverändert übertragen. Durch die Funktion `printf` wird diese Bitfolge nur wieder als `int`-Wert interpretiert – dass das nicht funktionieren kann, sollte klar sein; was man erhält, lässt sich dagegen allgemein *nicht* vorhersagen. Vielleicht probieren Sie es selbst einmal aus.

Hinter der Typbindung der Zeiger steht aber nicht nur die Notwendigkeit, ggf. Typen umzuwandeln. Auch für Operationen mit den Zeigern selbst wird der Typ vielfach benötigt. Um bei Zeigerarithmetik eine konkrete Adresse im Speicher zu berechnen, wird die Größe des entsprechenden Typs benötigt. Beim Inkrementieren eines Zeigers muss die Adresse ggf. um *mehrere* Bytes erhöht werden, damit der Zeiger auf das nächste Element zeigt. Durch die Bindung des Typs an den Zeiger steht auch diese Information zur Verfügung.

Ein Beispiel: Auf gängigen Maschinen braucht man mehrere Bytes um einen `int`-Wert zu speichern - z.B. auf 32-Bit Maschinen genau vier. Bei dem Vektor

```
int v[10];
```

ist die Speicheradresse von `v[1]` also um 4 größer als die Speicheradresse von `v[0]` und nicht nur um 1. In der Regel ist das kein Problem, weil der Compiler bei allen Zeigeroperationen den Speicherbedarf des jeweiligen Typs berücksichtigt. Schreiben wir aber etwa

```
int v[10];
char *s1, *s2;
ptrdiff_t i;
...
s1 = (char *) &v[0];
s2 = (char *) &v[1];
i = s2 - s1;
```

so erhält `i` allenfalls auf Rechnern mit sehr ausgefallener Organisation den Wert 1; in der Regel wird der Wert 2 oder 4 sein, je nachdem, wie viele Bytes der Rechner für einen `int`-Wert verwendet.

12.10 Zeiger und Felder (II)

Verschiedentlich wurden in Variablendefinitionen bereits Anfangswertzuweisungen für Zeigervariablen verwendet, ohne darauf besonders einzugehen. Daraus können Sie entnehmen, dass dieses ohne weiteres möglich ist. Trotzdem gibt es einige Dinge zu beachten, die sich insbesondere bei Strings auswirken.

Am Ende des letzten Abschnitts hatten wir vier weitgehend äquivalente Möglichkeiten kennengelernt, einer Stringvariablen einen Anfangswert zu geben:

```
char str1[6] = "hallo",
    str2[6] = {'h', 'a', 'l', 'l', 'o', '\0'},
    str3[] = "hallo",
    str4[] = {'h', 'a', 'l', 'l', 'o', '\0'};
```

Hierdurch werden, wohlgemerkt, nur Anfangswerte festgelegt. Wir dürfen also ohne weiteres

```
str1[1] = 'o';
str1[4] = 'a';
```

schreiben, wodurch der Wert zu "holla" wird.

Eine letztlich durchaus andere Wirkung hat die ebenfalls zulässige Anfangswertzuweisung

```
char *str5 = "hallo";
```

Hier bezeichnet `str5` nicht mehr den Vektor, in dem der String gespeichert wird, sondern eine Variable, deren Anfangswert der Zeiger auf den Anfang des Vektors ist.

Während die Wertzuweisung

```
str1 = str5;
```

unzulässig ist, weil der Name eines Feldes eine Zeigerkonstante ist und entsprechend *nicht* verändert werden kann, darf man

```
str5 = str1;
```

schreiben – der Wert einer Zeigervariablen darf natürlich jederzeit verändert werden. Ob das allerdings vernünftig ist, ist eine andere Frage: Wenn wir im Beispiel den Wert von `str5` verändern, ohne dass wir den Anfangswert zuvor in eine andere Zeigervariable umspeichern, haben wir in der Folge *keinen Zugriff* mehr auf den String (auch wenn er natürlich unverändert im Speicher steht)! Solche Zuweisungen sind also, obwohl formal zulässig, logisch ein Fehler, von seltenen Ausnahmen vielleicht abgesehen.

Zweckmäßig ist es also, solche Zeigervariablen mit dem Attribut `const` zu versehen:

```
char *const str5 = "hallo";
```

Jetzt darf der Wert von `str5` im Programm nicht mehr verändert werden.

Entsprechend kann und sollte man `const` verwenden, wenn die Strings selbst unveränderbare Konstanten sein sollen. Typische Formulierungen sind zum Beispiel

```
const char str3[] = "hallo";
const char *const str5 = "hallo";
```

Alles hier gesagte, gilt für Vektoren und Zeiger mit anderen Typen natürlich in gleicher Weise.

Eines müssen wir an dieser Stelle aber noch klären: Was bedeutet bzw. bewirkt eine Wertzuweisung wie

```
s = "hallo";
```

C kennt keine Wertzuweisung, bei der der Wert eines String (oder allgemeiner: der Wert eines Feldes) übertragen wird! Entsprechend muss der String hier faktisch als Zeiger auf den Anfang der Zeichenfolge interpretiert werden, wobei die Zeichenfolge selbst vom Compiler irgendwo im Speicher abgelegt wird. Die Variable *s* muss damit den Typ `char *` besitzen!

12.11 Zeigervektoren

Vergleicht man die letzten beiden Anfangswert-Beispiele

```
const char str3[] = "hallo";
const char *const str5 = "hallo";
```

genauer, so stellt man fest, dass die erste Formulierung vorzuziehen ist: `str3` und `str5` repräsentieren zwar letztlich beide konstante Zeiger auf konstante Strings, die Behandlung ist jedoch durchaus verschieden:

- `str3` ist formal eine Konstante: Sie besitzt keinen Speicherplatz, sondern zeigt selbst auf den Anfang des konstanten String.
- `str5` ist eine Variable: Sie besitzt einen Speicherplatz, so dass erst ihre Dereferenzierung den Zeiger auf den Anfang des konstanten Strings liefert.

In bestimmten Zusammenhängen können Zeigervariablen mit konstanten Werten durchaus sinnvoll sein. Wir wollen uns das an einem etwas ausführlicheren Beispiel ansehen: Auf Zahlungsanweisungen und anderen Belegen müssen Beträge in Buchstaben wiederholt werden. Man behilft sich gelegentlich damit, die Ziffern einzeln in Buchstaben umzusetzen und irgendein Trennzeichen dazwischen zu setzen, so dass etwa 138 die Zeichenfolge

```
eins = drei = acht
```

ergibt.

Die Buchstabenfolgen wird man in einem Feld speichern. Wir können etwa definieren

```
const char ziffern[][7] = {
    "null", "eins", "zwei", "drei", "vier",
    "fuenf", "sechs", "sieben", "acht", "neun"
};
```

Dabei „verschwendet“ man allerdings Speicherplatz: Jede Zeile der Matrix kann 7 Zeichen aufnehmen; benötigt werden diese 7 Zeichen aber nur für die **sieben**, während für 7 Buchstabenfolgen jeweils 5 Zeichen ausreichen. Hier spielt das keine große Rolle – dass wir 70 Bytes für die Matrix bereitstellen, obwohl wir nur 54 Bytes wirklich brauchen, macht nichts aus. Aber es geht um das Prinzip: Bei einer Folge von 100 Strings, von denen einer 99 Zeichen, die übrigen dagegen nur jeweils 4 Zeichen lang sind, macht es schon einen Unterschied, ob wir eine (100×100) -Matrix definieren und damit 10000 Bytes belegen, oder ob wir nur die benötigten 595 Bytes für die Strings verwenden.

Prinzipiell kann man das Problem lösen, indem man einen Vektor von Zeigern auf Strings definiert:

```
const char *const ziffern[] = {
    "null", "eins", "zwei", "drei", "vier",
    "fuenf", "sechs", "sieben", "acht", "neun"
};
```

Jetzt ist jede Komponente des Vektors ein Zeiger auf einen String; die Strings selbst speichert der Compiler irgendwo – mit genau der Länge, die sie einschließlich des abschließenden Null-Zeichens haben. Es soll nicht verschwiegen werden, dass diese Lösung im konkreten Beispiel aufwendiger ist als die erste Formulierung: Bei den Strings sparen wir zwar 16 Bytes ein – dafür definieren wir den Zeigervektor zusätzlich, der, je nach Rechner, in der Regel 20 oder 40 Bytes benötigen wird!

Nach diesen Vorüberlegungen zu den Daten erweist sich die Realisierung als einfach: Eine rekursive Funktion erledigt die eigentliche Arbeit. Sie wird in einen nichtrekursiven Rahmen eingebettet, weil der Trennstring nur zwischen je zwei Ziffern geschrieben werden darf, der dafür erforderliche zusätzliche Parameter für den Benutzer der Funktion aber „versteckt“ werden soll:

```
void umsetzen (int wert) {
    um_rek (wert, FALSE);
}

void um_rek (int wert, int trennen) {
    if (wert > 10)
        um_rek (wert / 10, TRUE);
    printf ("%s", ziffern[wert % 10]);
    if (trennen)
        printf (" = ");
}
```

12.12 Zeiger auf Zeiger

Mit den Daten des letzten Beispiels haben wir eine weitere Möglichkeit kennengelernt, nämlich Zeiger auf Zeiger: Der Name des Vektors ist der Zeiger auf den Anfang einer Folge von Variablen, die ihrerseits Zeiger auf Strings sind. Auch solche Variablen kann man explizit definieren: Durch

```
char **tabelle;
```

ist `tabelle` ein Zeiger auf eine Variable mit dem Typ `char *`, also Zeiger auf eine Variable mit einem Zeigertyp.

Dass Zeiger auf Zeiger keine theoretische Spielerei sind, kann durch eine kleine Erweiterung des Beispiels „Zahlen in Klarschrift“ gezeigt werden: Die Klarschrift soll wahlweise in Deutsch oder Englisch (oder einer beliebigen anderen Sprache) erfolgen. Die Entscheidung darüber soll erst bei der Ausführung des Programms getroffen werden.

Im wesentlichen müssen wir die Datendefinition erweitern:

```
const char *const ziffern_d[] = {
    "null", "eins", "zwei", "drei", "vier",
    "fuenf", "sechs", "sieben", "acht", "neun"
};

const char *const ziffern_e[] = {
    "zero", "one", "two", "three", "four",
    "five", "six", "seven", "eight", "nine"
};

const char *const *ziffern;
```

Im Programm brauchen wir jetzt nur noch dafür zu sorgen, dass `ziffern` der „richtige“ Zeiger zugewiesen wird, zum Beispiel durch

```

if (...)
    ziffern = ziffern_d;
else
    ziffern = ziffern_e;

```

Generell entsprechen mehrdimensionale Felder Zeigern auf Zeiger, wie bereits kurz angesprochen. Haben wir zum Beispiel die Definition

```
int m[2][3];
```

so ist `m` Zeiger auf den Vektor der Zeilenvektoren, während `m[i]`, einen „zulässigen“ Wert von `i` vorausgesetzt, der Zeiger auf das erste Element in der $(i + 1)$ -ten Zeile der Matrix ist. Damit haben wir, zulässige Werte der Indizes vorausgesetzt, vier Möglichkeiten, auf ein bestimmtes Element zuzugreifen, nämlich

```

m[i][j]
*(m + i)[j]
*(m[i] + j)
*(*(m + i) + j)

```

12.13 Zeiger als Funktionswerte

Auch wenn es selbstverständlich sein sollte: Funktionen können als Funktionswert einen Zeiger liefern.

Speziell bei Funktionen zur Verarbeitung von Strings ist der Funktionswert häufig ein Zeiger auf einen (Teil-)String. Das ist auch der Grund, warum auf die Funktionen bislang nicht näher eingegangen wurde, die in der Standard-Headerdatei `string.h` deklariert sind. Deshalb jetzt ein kleiner Nachtrag zu dieser Header-Datei.

Zum Kopieren von Strings stehen die beiden Funktionen

```

char *strcpy(char *s1, const char *s2);
char *strncpy(char *s1, const char *s2, size_t n);

```

zur Verfügung: `strcpy` kopiert so lange, bis das Null-Zeichen von `s2` übertragen wurde. `strncpy` schreibt dagegen genau `n` Zeichen, wobei ggf. am Ende Nullen ergänzt werden. Der Funktionswert ist der Zeiger auf `s1`. Bei beiden Funktionen dürfen sich Quellstring `s2` und Zielstring `s1` nicht überschneiden; dass der Zielstring genug Platz für den Quellstring bietet, liegt ausschließlich in der Verantwortung des Programmierers.

Der Typ `size_t` ist übrigens ein vorzeichenloser ganzzahliger Typ, der in `stddef.h`, `stdlib.h` und `string.h` durch `typedef` deklariert ist.

In analoger Weise kopieren die Funktionen

```

char *strcat(char *s1, const char *s2);
char *strncat(char *s1, const char *s2, size_t n);

```

den String `s2` hinter den String `s1`. Man bezeichnet das als *Konkatenation*. Dass der Zielstring genug Platz bietet, liegt ausschließlich in der Verantwortung des Programmierers.

Ebenfalls analog erlauben die Funktionen

```

int strcmp(const char *s1, const char *s2);
int strncmp(const char *s1, const char *s2, size_t n);

```

den Vergleich von zwei Strings. Der Funktionswert ist kleiner, gleich oder größer als Null, je nachdem, ob `s1` kleiner, gleich oder größer als `s2` ist. Beim Vergleich werden die Ordnungszahlen der Zeichen verwendet, so dass Groß- und Kleinbuchstaben als verschieden betrachtet werden.

Verschiedene, teils ziemlich komplizierte Funktionen erlauben das Suchen von Zeichen oder Strings in Strings. Die wichtigsten dieser Funktionen sind:

```
char *strchr(const char *s, int c);
char *strrchr(const char *s, int c);
char *strstr(const char *s1, const char *s2);
```

Die ersten beiden Funktionen suchen das Zeichen `c` im String `str` und liefern als Funktionswert den Zeiger auf die erste gefundene Position oder den Nullzeiger, wenn `c` in `s` nicht vorkommt. Während `strchr` die Suche am Anfang von `s` startet, startet `strrchr` am Ende von `s`. `strstr` sucht die Zeichenfolge von `s2` in `s1`. Der Funktionswert ist der Zeiger auf das erste Zeichen der gefundenen Teilfolge bzw. der Nullzeiger.

Schließlich gibt es die von uns bereits betrachtete Funktion

```
size_t strlen(const char *s);
```

zur Bestimmung der aktuellen Länge des String `s`.

Als Beispiel soll einmal die Funktion `strchr` implementiert werden:

```
char *strchr(const char *s, char c) {
    while (*s != c)
        if (*s++ == '\0')
            return NULL;
    return s;
}
```

Hierbei ist anzumerken, dass der Compiler bei der `return`-Anweisung warnen sollte, dass das `const`-Attribut verlorengeht. Als Rückgabewert erhält man ja einen Zeiger auf *nicht*-konstante Werte. Folgende zwei Zeilen sollten ohne Warnung übersetzt werden können:

```
char *p = strchr("Konstanter Strung", 'u');
*p = 'i';
```

Das Ergebnis bei der Ausführung ist nicht definiert. Beim Argument von `strchr` handelt es sich schließlich um eine Zeichenketten*konstante*, auf die auch der zurückgegebene Zeiger verweist.

12.14 Parameter des Hauptprogramms

Jedes C-Programm muss, wie wir wissen, mit der Funktion `main` ein Hauptprogramm besitzen. Diese Hauptfunktion wird beim Start des Programms vom Betriebssystem aufgerufen. Und sie erhält beim Aufruf zwei Argumente! Wir haben diese Argumente bislang durch die Definition

```
int main (void)
```

ignoriert. Tatsächlich ist der Prototyp für `main` jedoch

```
int main (int argc, char *argv[]);
```

Was man übergeben erhält, ist der bereits zerlegte Inhalt der Aufrufzeile des Programms:

- `argc` ist die Anzahl der Einträge in der Aufrufzeile, wobei als Trennzeichen zwischen den Einträgen Leerzeichen und horizontaler Tabulator interpretiert werden.
- `argv` ist ein Vektor von Zeigern auf die einzelnen Einträge in der Form von C-Strings.

Der String, auf den `argv[0]` zeigt, ist der Name des Programms oder ein leerer String, wenn das Betriebssystem den Namen des Programms nicht liefert. Die nächsten `argc - 1` Komponenten zeigen auf die weiteren Einträge der Aufrufzeile, in der Reihenfolge, in der sie in der Aufrufzeile angegeben sind. Dem letzten „echten“ Zeiger folgt noch eine Komponente mit dem Nullzeiger (`argv[argc]`), so dass man sich wahlweise an diesem Nullzeiger oder an dem Wert von `argc` orientieren kann.

Nicht übergeben werden dabei alle Einträge der Aufrufzeile, die sich auf die Umleitung der Ein- und Ausgabe oder auf Filter beziehen – das sollte aber auch klar sein, da dieses vom Betriebssystem bearbeitet wird und für das Programm transparent sein soll.

Was ein Programm mit seinen Parametern anfängt, ist ihm selbst überlassen. Es kann sie ignorieren, wie wir es bislang getan haben, oder aber auch darauf reagieren.

Wir betrachten zunächst ein eher formales Beispiel: Ein Programm soll die Einträge seiner Aufrufzeile auflisten. Da der Nullzeiger hier erstmals für uns Bedeutung gewinnt, wird er hier verwendet, um das Ende der Liste der Einträge abzufangen. Die Realisierung kann dann so aussehen:

```
int main(int argc, char *argv[]) {
    while (*++argv != NULL)
        printf("%s\n", *argv);
    return 0;
}
```

Ein realistischeres Beispiel könnte so aussehen: Wir hatten die Umsetzung von Zahlen in Klarschrift programmiert, in der letzten Version mit der Möglichkeit, die Klarschrift auf deutsch oder englisch zu erhalten. Es liegt nahe, die gewünschte Sprache nicht abzufragen, sondern als Parameter in der Aufrufzeile zu verlangen, zum Beispiel `-Sd` für deutsch und `-Se` für englisch. Man kann dann weiter festlegen, dass bei fehlendem Parameter `-Sx` wie bei `-Sd` verfahren wird und dass bei `-Sx` mit einem anderen Zeichen als `d` und `e` das Programm mit einer Fehlermeldung abgebrochen wird. Ebenso sollen mehr als 2 Parameter zum Programmabbruch führen.

Die Realisierung des Hauptprogramms kann dann so aussehen:

```
#include <string.h>

enum sprachen {DEUTSCH, ENGLISCH};

int main(int argc, char *argv[]) {
    enum sprachen sprache = DEUTSCH;

    if (argc > 2) {
        printf ("Zu viele Parameter!\n");
        return 1;
    }

    if (argc == 2) {
        if (strcmp(argv[1], "-Se") == 0) {
            printf("Sprache Englisch gewaehlt.\n");
            sprache = ENGLISCH;
        }
        else if (strcmp(argv[1], "-Sd") == 0) {
            printf("Sprache Deutsch gewaehlt.\n");
            sprache = DEUTSCH;
        }
        else {
```

```

        printf("Unzulaessiger Parameter.\n");
        return 2;
    }
}
...
return 0;
}

```

12.15 Zeiger auf Funktionen

C erlaubt nicht nur Zeiger auf Variablen, sondern ebenso Zeiger auf Funktionen – auch *Funktionszeiger* genannt.

Zeiger auf Funktionen benötigt man zum Beispiel dann, wenn man Funktionen als Parameter an Funktionen übergeben will. Das ist nichts „Exotisches“:

- In der Mathematik hat man sehr häufig Algorithmen, die für ganze Funktionenklassen definiert sind. Ein „klassisches“ Beispiel hierfür ist die numerische Integration; eine Integrationsfunktion soll ja für beliebige integrierbare Funktionen verwendet werden können.
- Häufig hat man bei allgemein formulierten Sortier- und Suchfunktionen Funktionen als Parameter. Führt man zum Beispiel Vergleiche nicht explizit aus, sondern sieht eine Vergleichsfunktion vor, so kann man ohne Änderung an der eigentlichen Sortierfunktion nach beliebigen Kriterien sortieren.

Sehen wir uns aber zunächst die Formalien an: Die Definition eines Zeigers auf eine Funktion ähnelt der Deklaration einer Funktion:

```
funktionstyp (*name)(parametertyp, ..., parametertyp);
```

So ist durch

```
int (*fptr)(char);
```

die Variable `fptr` ein Zeiger auf eine Funktion, die einen `char`-Parameter besitzt und einen `int`-Funktionswert liefert. Die Klammern um `*fptr` sind hier zwingend notwendig, denn durch

```
int *fptr (char);
```

würde deklariert, dass `fptr` selbst eine Funktion ist, die einen `char`-Parameter besitzt und als Funktionswert einen Zeiger auf `int` liefert! Die Deklaration von Funktionszeigern verlangt einem also einiges an Schreibarbeit ab – besonders wenn man mehrere von ihnen braucht. Hier kommt das Schlüsselwort `typedef` gelegen. Dazu betrachten wir folgendes Beispiel:

```

#include <stdio.h>

typedef double (*fptr_t)(double, double);

double summe(double a, double b) {
    return a + b;
}

double produkt(double a, double b) {
    return a * b;
}

```

```

/*= Hauptprogramm =====*/
int main (void) {
    /* Feld von Funktionszeigern */
    fptr_t vfptr[2] = { summe, produkt };
    unsigned int op = 0;
    double a = 0.0, b = 0.0;

    printf("Eingabe: Operator Argument1 Argument2\n"
           "Operatoren: 0 = Summe, 1 = Produkt\n");
    scanf("%u %lf %lf", &op, &a, &b);

    printf("Ergebnis: %f\n", vfptr[op](a, b));

    return 0;
}

```

Quelltext 12.1: Verwendung von Zeigern auf Funktionen und typedef

Durch die Typ-Deklaration von `fptr_t` spart man bei der weiteren Verwendung das wiederholte Ausschreiben des Funktionszeiger-Typs. Selbst die Deklaration eines Feldes von Zeigern auf Funktionen ist dann nicht schwieriger als z.B. beim Typ `int`. Als Ausgabe des Programms erhalten wir

```

Eingabe: Operator Argument1 Argument2
Operatoren: 0 = Summe, 1 = Produkt
1
42
23
Ergebnis: 966.000000

```

Bemerkenswert an diesem kleinen Beispiel ist, dass die Auswahl der gewünschten Operation ohne `if`-Anweisung oder ähnliches auskommt. Wie für Felder ist auch für Funktionen der Adressoperator nicht erforderlich: Wenn eine Funktion aufgerufen werden soll, muss dem Namen die Argumentliste folgen, also mindestens ein leeres Klammernpaar. Steht der Name einer Funktion ohne Argumentliste im Quellcode, dann ist der Zeiger auf sie gemeint.

Als konkretes Beispiel für Funktionszeiger als Argument soll die Funktion `qsort` dienen, die in der Standard-Headerdatei `stdlib.h` deklariert ist. Ihr Prototyp ist

```

void qsort(void *start, size_t anzahl, size_t groesse,
           int (*relation)(const void *, const void *) );

```

Diese Funktion sortiert grundsätzlich aufsteigend – nur kann und muss ihr Anwender mit einer eigenen Relationsfunktion festlegen, was „aufsteigend“ bedeutet. Daneben unterstellt die Funktion, dass der Vektor zu sortieren ist, den die ersten drei Argumente beschreiben:

- Das erste Argument ist der Zeiger auf die erste Komponente des Vektors.
- Das zweite Argument ist die Anzahl der Komponenten des Vektors.
- Das dritte Argument ist die Größe der Komponenten des Vektors.

Diese Formulierung, insbesondere die Verwendung von `void *`, erlaubt es, beliebige Vektoren als Argumente zu übergeben. Man beachte dabei: Variablen, Parameter und Funktionswerte dürfen zwar den Typ `void *` erhalten; dereferenzieren kann man solche Werte allerdings erst, nachdem man sie mit einem Castoperator in einen typgebundenen Zeiger

umgewandelt hat. Dass man dabei nur „sinnvolle“ Umwandlungen vornehmen darf, sollte klar sein, kann jedoch vom Compiler nicht überprüft werden.

Der Name der Funktion legt nahe, dass sie Quicksort realisiert – aber das ist nicht vorgeschrieben.

Jetzt zum Beispiel: Wir wollen `short`-Zahlen auf- oder absteigend sortieren. Dann können wir schreiben

```
#include <stdlib.h>

#define LAENGE ...

typedef int (*REL)(const void *, const void *);

int aufsteigend(const short *const links,
               const short *const rechts);
int absteigend(const short *const links,
               const short *const rechts);

int main(void) {
    short v[LAENGE];
    int (*rel)(const short *const,
               const short *const);
    ...
    if (...)
        rel = aufsteigend;
    else
        rel = absteigend;
    ...
    qsort((void *) v, (size_t) LAENGE,
          sizeof (short), (REL) rel);
    ...
    return 0;
}

int aufsteigend(const short *const links,
               const short *const rechts) {
    return *links - *rechts;
}

int absteigend(const short *const links,
               const short *const rechts) {
    return *rechts - *links;
}
```

Damit sollte das Prinzip klar sein, auch für Vektoren mit anderen Typen. Der Castoperator für die Vergleichsfunktion mag auf den ersten Blick ziemlich „exotisch“ erscheinen, aber er ist ohne weiteres zulässig und wandelt den Typ von `rel` entsprechend dem Prototyp von `qsort` um.

Man kann sich nun leicht überlegen, dass man selbst die „merkwürdigsten“ Sortierreihenfolgen leicht realisieren kann, zum Beispiel aufsteigend nach Beträgen oder erst alle geraden Werte aufsteigend und dann alle ungeraden Werte absteigend. Aber das überlegen Sie sich in den Details vielleicht selbst einmal.

12.16 Kopieren und umspeichern von Speicherblöcken

Da die Funktion `qsort` mit `void`-Zeigern arbeitet, besitzt sie keine Information über den Typ der zu sortierenden Daten. Die Zeiger können also auch nicht dereferenziert und die Elemente durch den Zuweisungsoperator vertauscht werden. Stattdessen werden sie einfach als binäre Daten kopiert. Diese Aufgabe übernehmen auch die folgende beiden Funktionen aus der Standardbibliothek.

```
void *memcpy(void *s1, const void *s2, size_t n);
void *memmove(void *s1, const void *s2, size_t n);
```

Beide kopieren `n` aufeinanderfolgende Zeichen, auf deren erstes `s2` zeigt, in den Speicherbereich, auf dessen Anfang `s1` zeigt.

Die Intention dieser beiden Funktionen ist letztlich jedoch nicht, wie `strcpy` und `strncpy` Zeichen zu kopieren, sondern beliebige zusammenhängende Speicherbereiche, auch *Speicherblöcke* genannt. Diese werden nur temporär als Zeichenvektoren betrachtet. Entsprechend werden die Werte der „Zeichen“ auch nicht untersucht, so dass ein eventuelles Null-Zeichen das Kopieren *nicht* beendet.

Unterschiede bestehen zwischen den beiden Funktionen, wenn Speicherbereiche kopiert werden sollen, die sich überlappen. Während `memmove` auch in solch einem Fall korrekt arbeitet, ist das bei `memcpy` nicht garantiert.

Beispiel:

```
#define LAENGE ???
...
double v1[LAENGE], v2[LAENGE];

/* korrekt: */
memcpy(v1, v2, sizeof v2);
memcpy(v1, v2, LAENGE * sizeof (double));
/* fraglich: */
memcpy(v1, v1+1, (LAENGE - 1) * sizeof (double));

/* korrekt: */
memmove(v1, v2, sizeof v2);
memmove(v1, v2, LAENGE * sizeof (double));
memmove(v1, v1+1, (LAENGE - 1) * sizeof (double));
```

Mit der gleichen Logik arbeitet die Funktion

```
void memset(void *v, int c, size_t n);
```

Sie schreibt den Wert von `c`, umgewandelt in den Typ `char`, `n`-mal in den Vektor, auf dessen Anfang `v` zeigt.

